Task1

```
!pip install pillow
```

```
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
```

```
from PIL import Image
```

```
image = Image.open("/content/drive/MyDrive/AI and ML/golden.jpeg")
print(f"Height: {image.height}, Width: {image.width}, Format: {image.format}, Mode: {image.mode}")
```

```
Height: 194, Width: 259, Format: JPEG, Mode: RGB
```

```
import matplotlib.pyplot as plt

plt.imshow(image)
plt.axis('off')
plt.show()
```

```
image_gray = image.convert('L')
display(image_gray)
```

```
width, height = image.size
channels = len(image_gray.getbands())
print(f"Image shape (gray): ({height}, {width}, {channels})")
image_size_grayed = width*height*1
print(f"Image size gray: {image_size_grayed}")

print(f"Width: {width}, Height: {height}")
```

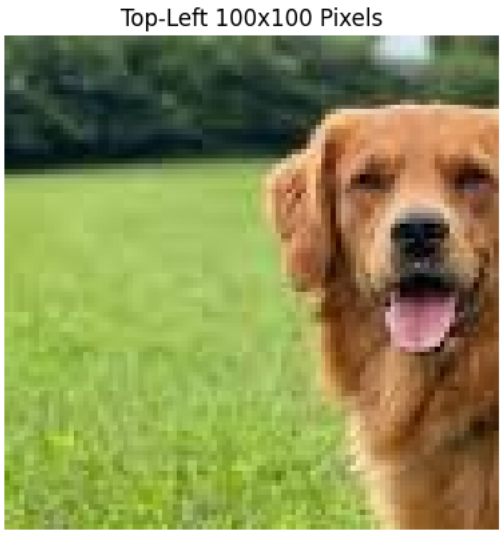
```
Image shape (gray): (194, 259, 1)
Image size gray: 50246
Width: 259, Height: 194
```

Start coding or generate with AI.

Extract top 100x100 pixels

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Convert the PIL image to a NumPy array
image_np = np.array(image)

# Extract the top-left 100x100 pixels
top_left_corner = image_np[0:100, 0:100]

# Display the extracted region
plt.imshow(top_left_corner)
plt.title('Top-Left 100x100 Pixels')
plt.axis('off')
plt.show()
```



Separate the image into its three color channels (Red, Green, and Blue) and display them individually, labeling each channel as R, G, and B.{Using NumPy.}

```
import matplotlib.pyplot as plt


# Separate the channels
red_channel = image_np.copy()
red_channel[:, :, 1] = 0
red_channel[:, :, 2] = 0
green_channel = image_np.copy()
green_channel[:, :, 0] = 0
green_channel[:, :, 2] = 0
blue_channel = image_np.copy()
blue_channel[:, :, 0] = 0
blue_channel[:, :, 1] = 0

# Display each channel
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(red_channel)
plt.title('Red Channel')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(green_channel)
plt.title('Green Channel')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(blue_channel)
plt.title('Blue Channel')
plt.axis('off')

plt.tight_layout()
plt.show()
```

4. Modify the top 100 × 100 pixels to a value of 210 and display the resulting image: • Modify the pixel values of the top-left 100 × 100 region to have a value of 210 (which is a light gray color), and then display the modified image.

```
mod_img = image_np.copy()
# Modify the top-left 100x100 region to a value of 210
mod_img[0:100, 0:100] = 210

# Display the modified image
plt.figure(figsize=(8, 8))
plt.imshow(mod_img)
plt.title('Image with Top-Left 100x100 Pixels Modified to 210')
plt.axis('off')
plt.show()
```

Task2

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img = Image.open("/content/drive/MyDrive/AI and ML/golden.jpeg").convert("L")
img_np = np.array(img)

plt.imshow(img_np, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

height, width = img_np.shape
center_y = height // 2
center_x = width // 2

cropped = img_np[center_y-75:center_y+75, center_x-75:center_x+75]
```

```
plt.imshow(cropped, cmap='gray')
plt.title("Center 150x150 Section")
plt.axis("off")
plt.show()

binary_img = img_np.copy()
binary_img[binary_img < 100] = 0
binary_img[binary_img >= 100] = 255

plt.imshow(binary_img, cmap='gray')
plt.title("Binary Image")
plt.axis("off")
plt.show()

rotated = img.rotate(-90)
rotated_np = np.array(rotated)

plt.imshow(rotated_np, cmap='gray')
plt.title("Rotated Image")
plt.axis("off")
plt.show()

rgb_img = np.stack((img_np, img_np, img_np), axis=-1)

plt.imshow(rgb_img)
plt.title("RGB Image")
plt.axis("off")
plt.show()
```

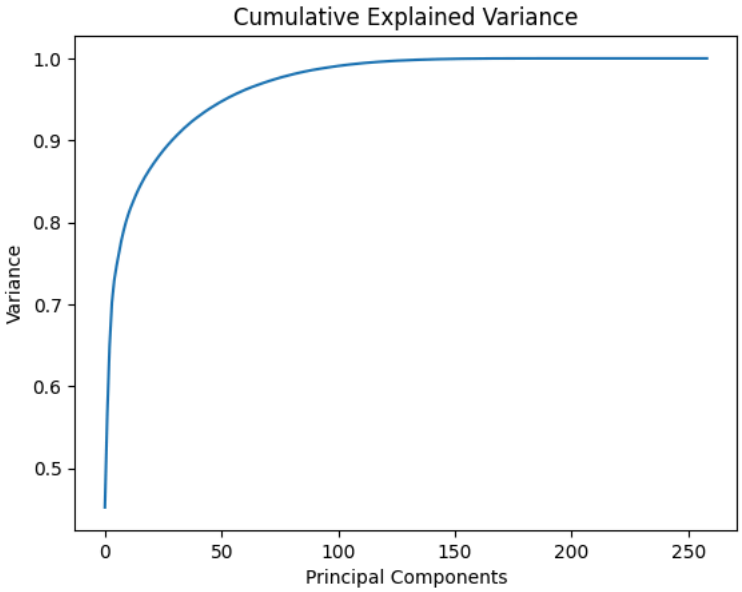
Grayscale Image

Center 150x150 Section

Task3

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img = Image.open("/content/drive/MyDrive/AI and ML/golden.jpeg").convert("L")
img_np = np.array(img)

plt.imshow(img_np, cmap='gray')
plt.title("Original Image")
plt.axis("off")
plt.show()

mean = np.mean(img_np, axis=0)
centered = img_np - mean

cov_matrix = np.cov(centered, rowvar=False)

eigen_values, eigen_vectors = np.linalg.eig(cov_matrix)

sorted_index = np.argsort(eigen_values)[::-1]
eigen_values = eigen_values[sorted_index]
eigen_vectors = eigen_vectors[:, sorted_index]

# take real part
eigen_vectors = np.real(eigen_vectors)

explained_variance = eigen_values / np.sum(eigen_values)
cumulative_variance = np.cumsum(explained_variance)

plt.plot(cumulative_variance)
plt.title("Cumulative Explained Variance")
plt.xlabel("Principal Components")
plt.ylabel("Variance")
plt.show()

k_values = [5, 20, 50, 100]

plt.figure(figsize=(10,8))

for i, k in enumerate(k_values):
    pcs = eigen_vectors[:, :k]
    compressed = np.dot(centered, pcs)
    reconstructed = np.dot(compressed, pcs.T)
    reconstructed = reconstructed + mean
    reconstructed = np.real(reconstructed)  # ensure real numbers for plotting

    plt.subplot(2,2,i+1)
    plt.imshow(reconstructed, cmap='gray')
    plt.title(str(k) + " PCs")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



5 PCs

20 PCs